

Gestão Ágil de Projetos de Software

Prof. Orlando Saraiva Júnior
orlando.nascimento@fatec.sp.gov.br



Como o cliente explicou...



Como o líder de projeto entendeu...



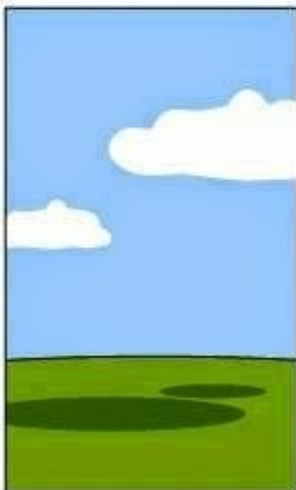
Como o analista projetou...



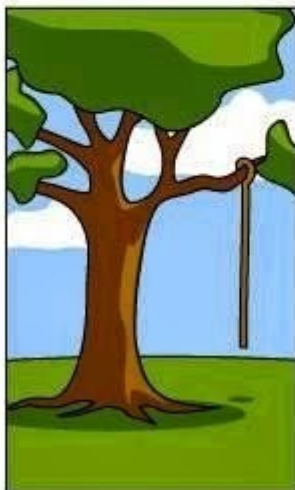
Como o programador construiu...



Como o Consultor de Negócios descreveu...



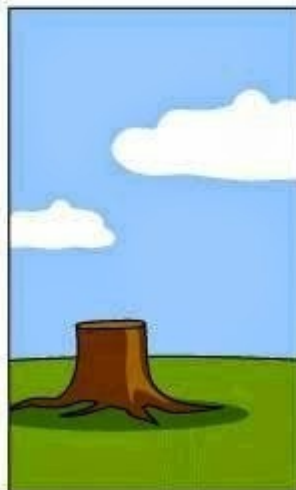
Como o projeto foi documentado...



Que funcionalidades foram instaladas...



Como o cliente foi cobrado...



Como foi mantido...



O que o cliente realmente queria...

Escrever programas é fácil mas e quando ...

Inúmeras pessoas escrevem programas.

Pessoas envolvidas com negócios escrevem programas em planilhas para simplificar seu trabalho; cientistas e engenheiros escrevem programas para processar seus dados experimentais; e há aqueles que escrevem programas como hobby, para seu próprio interesse e diversão.

... software é um atividade profissional ?

No entanto, a maior parte do desenvolvimento de software é uma atividade profissional, em que o software é desenvolvido para um propósito específico de negócio, para inclusão em outros dispositivos ou como produtos de software como sistemas de informação.

A engenharia de software tem por objetivo apoiar o desenvolvimento profissional de software, mais do que a programação individual.

Ela inclui técnicas que apoiam especificação, projeto e evolução de programas, que normalmente não são relevantes para o desenvolvimento de software pessoal.

Sintomas:

Projetos estourando o orçamento;

Projetos estourando o prazo;

Software de baixa qualidade;

Software muitas vezes não satisfaz os requisitos;

Projetos ingerenciáveis e código difícil de manter;

Soluções:

Análise econômica de sistemas de informação;

O uso de melhores técnicas, métodos e ferramentas;

A mudança de paradigma sobre o que é desenvolver software e como deveria ser feito;

Em outubro de 1968, cerca de 50 renomados Cientistas da Computação se reuniu durante uma semana em Garmisch, na Alemanha.

Nesta conferência foi cunhado o termo Engenharia de Software.

Por isso, a Conferência da OTAN é considerada o marco histórico de criação da área de Engenharia de Software.



Um processo de software é uma sequência de atividades que leva à produção de um produto de software. Existem quatro atividades fundamentais comuns a todos os processos de software.

Especificação de software, em que clientes e engenheiros definem o software a ser produzido e as restrições de sua operação.

Desenvolvimento de software, em que o software é projetado e programado.

Validação de software, em que o software é verificado para garantir que é o que o cliente quer.

Evolução de software, em que o software é modificado para refletir a mudança de requisitos do cliente e do mercado.

Análise e Planejamento Ágeis

Análise e Planejamento Ágeis

Análise e planejamento ágeis é o exame de um negócio ou qualquer aspecto dele (cultura, estrutura organizacional, processo, produtos e muito mais) para aprender o que precisa mudar e quando a fim de alcançar um resultado desejado em a contexto que valoriza muito a adaptabilidade, a resiliência e a inovação contínua e a entrega de valor.

Inclui analisar para quem o produto é destinado (às partes interessadas), definir seus requisitos, determinar quando os recursos serão entregues e estimar custos e recursos.

Análise e Planejamento Ágeis

1940s: Kanban for Manufacturing
1950s: Lean Thinking (Toyota)
1980's: Objectory process, Boehm's Spiral Model
1991: Continuous Integration (CI) by Grady Booch
1994: DSDM Atern
1996: Scrum first developed
Late 90s: BA begins to appear in the wild
1999: Extreme Programming, User Stories
2001: Agile Development with Scrum published
2001: Agile Manifesto; Agile Alliance formed
2002: Test-Driven Development
2003: Lean Software Development
2003: IIBA founded
2005: LeSS (Bas Vodde & Crag Larman)
2005: IIBA CBAP certification implemented; BABOK 1 released
2006: International Requirements Engineering Board (IREB) founded in Germany
2007: Kanban for Software Development
2009: DevOps
2009: Lean Startup
2010 Jez Humble and David Farley introduce Continuous Delivery (CD)
2011: SAFe, **Use Case 2.0**
2015: *PMI-PBA Certification introduced*
2015: **Agile Extension to the BABOK Guide V1.0** published by IIBA
2017: **Agile Extension to the BABOK Guide V2.0** published by IIBA and Agile Alliance
2017: PMI publishes *Guide to Business Analysis* and Agile Practice Guide
2018: Jeff Sutherland releases the Scrum@Scale Guide
2020: The Scrum Guide, 2020; SAFe 5.0

Análise e Planejamento Ágeis

Analistas de negócios começaram a aparecer no final da década de 1990, quando as empresas começaram a introduzir essa função em suas organizações de desenvolvimento de software.

O foco inicial dos analistas de negócios estava nos requisitos - especificamente, a descoberta, criação, comunicação, gerenciamento e validação de requisitos de software.

Hoje, a competência análise de negócios abrange um escopo muito mais amplo, incluindo análise empresarial e estratégica.

Análise e Planejamento Ágeis

Duas estruturas iterativas foram adicionadas em meados do final da década de 1990: Scrum e Extreme Programming (XP). Ambos usaram um abordagem de planejamento com caixa de tempo, onde todo o trabalho para uma próxima iteração (a caixa de tempo) é comprometido antecipadamente.

Hoje, o Scrum é um dos frameworks ágeis mais amplamente utilizados - e mesmo aqueles que não adotam o framework inteiro costumam usar conceitos e eventos popularizados pelo Scrum, como velocidade e standup diário.

O XP contribuiu com práticas, como programação em pares. Seu impacto mais significativo na análise e planejamento ágeis é o uso de histórias, estimativa de pontos de história e o Jogo de Planejamento.

Análise e Planejamento Ágeis

Esses vários precursores do desenvolvimento ágil se uniram em 2001, quando dezessete pessoas se encontraram nas montanhas de Utah e surgiram com O Manifesto Ágil e 12 Princípios por trás do Manifesto.

Mais tarde naquele ano, alguns de seus autores e outros formaram a Agile Alliance, uma organização sem fins lucrativos, para promover práticas ágeis.

Desde a redação do Manifesto, muitas outras estruturas e práticas foram adicionadas à família ágil.

Isso inclui o desenvolvimento orientado a testes (TDD) e sua extensão, o desenvolvimento orientado a testes de aceitação –ATDD) e o Scaled Agile Framework (SAFe) para organizações ágeis em escala, lançado em 2011.

Análise e Planejamento Ágeis

Em 2009 e 2010, foram introduzidas duas práticas essenciais que tornaram possível implantar atualizações com frequência sem sacrificar a confiabilidade: DevOps e entrega contínua.

Essas práticas permitiram que o desenvolvimento ágil cumprisse plenamente sua promessa, conforme declarado em seus princípios originais “satisfazer o cliente por meio da entrega antecipada e contínua de software valioso”.

Usando DevOps/práticas de entrega contínua, as melhorias do produto podem ser lançadas com segurança várias vezes ao dia, em vez de uma vez a cada dois ou três meses, como costumava ser o caso quando eu trabalhava na década de 1990, quando o RUP era predominante.

Manifiesto Ágil

Manifesto Ágil



Estamos descobrindo melhores maneiras de desenvolver software fazendo isso e ajudando outras pessoas a fazê-lo. Através deste trabalho chegamos ao valor:

Indivíduos e interações	sobre	processos e ferramentas
Software de trabalho	sobre	documentação abrangente
Colaboração com o cliente	sobre	negociação de contrato
Respondendo à mudança	sobre	seguindo um plano

Ou seja, embora haja valor nos itens à direita, valorizamos mais os itens à esquerda.

Valores Manifesto Ágil

O primeiro valor central do Manifesto é: “Indivíduos e interações sobre processos e ferramentas.” Em linha com esse valor, a comunicação e o gerenciamento ágeis de requisitos são leves, com preferência por interações diretas e pessoais com partes interessadas e desenvolvedores em vez de procedimentos formais.

A segunda linha dos valores do Manifesto “software funcional sobre documentação abrangente.” Se você é um analista de negócios em cascata em transição para o ágil, isso significa que estará escrevendo menos documentação de requisitos do que está acostumado. Porque os requisitos são capturados e comunicados pouco antes de serem necessários, você não precisa escrever tanto para evitar mal-entendidos.

Valores Manifesto Ágil



A terceira linha do Manifesto valoriza “colaboração do cliente em vez de negociação de contratos” Se você já foi analista de negócios em cascata, esse valor representa uma mudança fundamental na forma como você interage com provedores de soluções e partes interessadas do negócio.

Em vez de fornecer especificações abrangentes antecipadamente que podem ser usadas como base para compromissos contratuais, seu foco muda para ajudar empresas e desenvolvimento a colaborar durante todo o desenvolvimento na descoberta de requisitos.

Valores Manifesto Ágil

A quarta linha do Manifesto valoriza a “capacidade de responder facilmente a mudanças seguindo um plano.” Este valor tem um impacto profundo na gestão de mudanças de requisitos e no planejamento da implementação. Se você estiver trabalhando em um projeto em cascata, os requisitos e o plano de implementação serão congelados antes da implementação.

Quaisquer alterações posteriores a isso desencadeiam um processo formal de mudança. Em uma iniciativa ágil, você não congela os requisitos, e apenas um processo leve é necessário para alterá-los.

Princípios por trás do Manifesto Ágil



Nossa maior prioridade é satisfazer o cliente através de entrega antecipada e contínua de software valioso.

Aceite mudanças de requisitos, mesmo no final desenvolvimento. Processos ágeis aproveitam a mudança para a vantagem competitiva do cliente.

Entregue software funcional com frequência, a partir de um algumas semanas a alguns meses, com um preferência pelo prazo mais curto.

Empresários e desenvolvedores devem trabalhar juntos diariamente durante todo o projeto.

Princípios por trás do Manifesto Ágil



Crie projetos em torno de indivíduos motivados. Dê-lhes o ambiente e o apoio de que necessitam, e confie neles para realizar o trabalho.

O método mais eficiente e eficaz de transmitir informações para e dentro de um empreendimento equipe é conversa cara a cara.

O software funcional é a principal medida de progresso.

Processos ágeis promovem o desenvolvimento sustentável. Os patrocinadores, desenvolvedores e usuários devem ser capazes para manter um ritmo constante indefinidamente.

Princípios por trás do Manifesto Ágil



Atenção contínua à excelência técnica e um bom design aumenta a agilidade.

Simplicidade - a arte de maximizar a quantidade de trabalho não realizado - é essencial.

As melhores arquiteturas, requisitos e projetos emergem de equipes auto-organizadas.

Em intervalos regulares, a equipe reflete sobre como para se tornar mais eficaz, então sintoniza e ajusta seu comportamento em conformidade.

Estruturas Ágeis

As estruturas ágeis que você precisa conhecer e seu impacto na análise e no planejamento.

Tenha em mente que é mais provável que sua organização use um híbrido de melhores práticas de várias fontes, em vez de aderir estritamente a uma única estrutura.

Por exemplo, ele pode usar a abordagem do Kanban para planejamento baseado em fluxo, adicionar ferramentas Scrum, como reuniões diárias e gráficos de burndown para monitorar o progresso, e usar histórias do XP para especificar itens de trabalho.

Kanban

O Kanban foi desenvolvido na década de 1950 para fabricação de automóveis e agora é amplamente utilizado para desenvolvimento de software.

Por exemplo, o Kanban está listado como uma das vinte e duas ferramentas de desenvolvimento enxuto de software.

Kanban não inclui o conceito de iterações fixas, uma vez que os itens progridem peça por peça.

Um dos insights importantes do Kanban é que regras simples e dicas visuais podem efetivamente governar comportamentos sofisticados.

O processo Kanban ocorre da seguinte maneira: você representa cada item de trabalho (por exemplo, um recurso ou história) como um cartão em um quadro Kanban.

Você cria uma coluna, ou fila, para cada status que precisa ser diferenciado e rastreado.

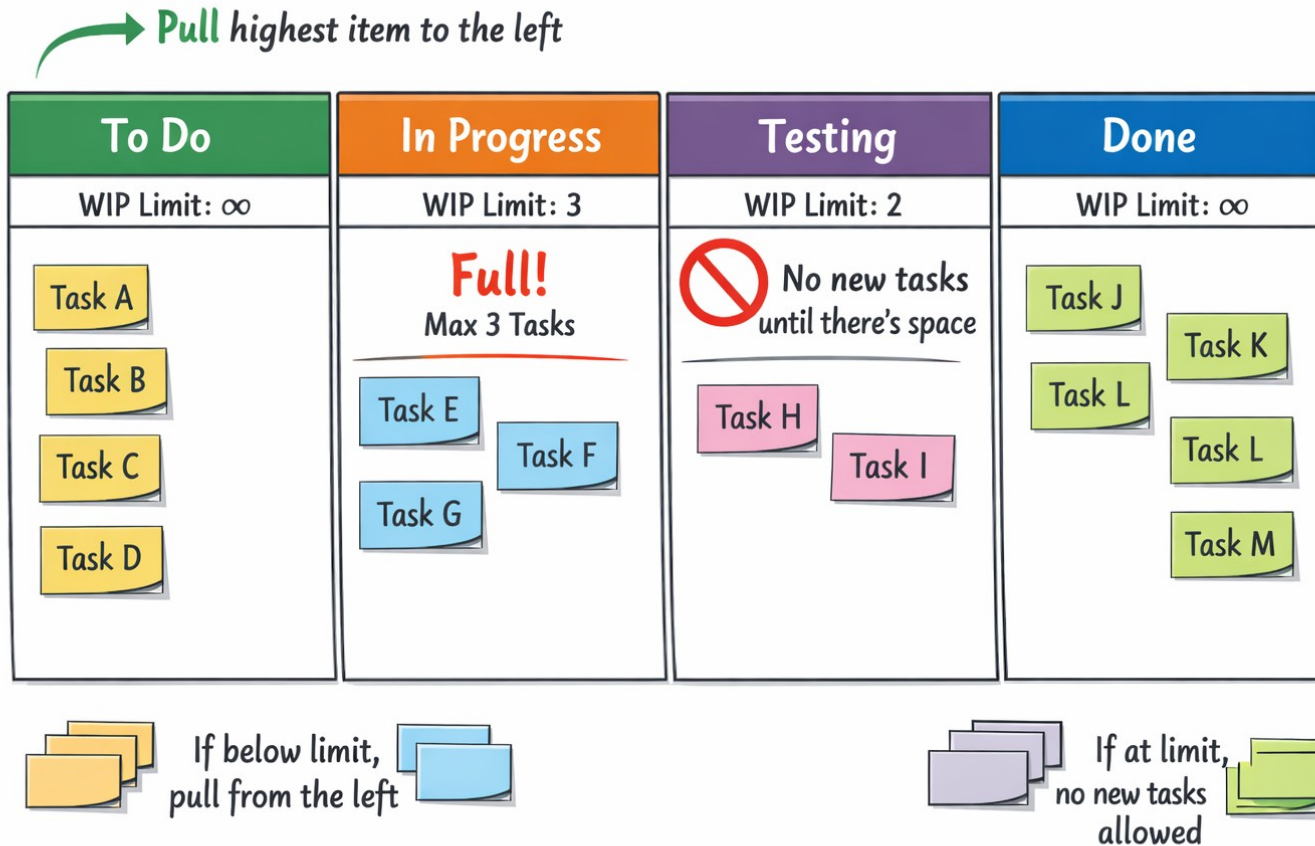
Em seguida, você move as cartas pelas colunas da esquerda para a direita, à medida que seu status muda. Você os move para cima ou para baixo dentro de uma coluna para indicar prioridade relativa.

O fluxo através das colunas é regido por algumas regras simples: Um limite de trabalho em andamento (WIP) é definido para cada fila.

Sempre que o número de itens em uma fila cai abaixo do limite do WIP, um desenvolvedor associado à fila puxa o item de trabalho mais alto da fila para a esquerda.

Quando o número de itens em uma fila atinge o limite do WIP, nenhum outro item pode ser adicionado à coluna do WIP até que um item tenha saído da fila.

Kanban



Scrum

O Scrum foi criado por Ken Schwaber e Jeff Sutherland e foi apresentado na conferência OOPSLA (Object-Oriented Programming, Systems, Languages, and Applications) em 1995.

Seus criadores descrevem o Scrum como “uma estrutura leve que ajuda pessoas, equipes e organizações a gerar valor por meio de soluções adaptativas para problemas complexos.”

A abordagem é definida no Guia Scrum, que você pode baixar gratuitamente em [scrum.org](https://www.scrum.org).

O Scrum é muito popular nos departamentos de TI das principais empresas. Sua popularidade provavelmente se deve a uma confluência de fatores:

- O processo Scrum fornece uma estrutura para planejamento e relatórios de status.
- O Scrum define uma cadência de desenvolvimento (de uma ou duas semanas a um mês) com a qual muitas organizações tradicionais podem conviver confortavelmente.
- Scrum tem um processo de certificação bem comercializado.

O Scrum está em desuso em alguns setores ágeis porque os itens são comprometidos no início de uma iteração (chamado sprint) e não mais tarde com base em prioridades mais atuais.

Apesar dessas objeções, o Scrum é uma excelente escolha para grandes itens de trabalho em nível de produto porque resulta em equipes livres ao mesmo tempo (o início de um sprint) para assumir compromissos.

Se você escolher Scrum, use timeboxing para controlar o fluxo de itens em uma iteração (sprint)—, mas não para atrasar o fluxo de itens por meio de integração e testes no final de seu ciclo de vida.

Mesmo que sua organização não use explicitamente a estrutura Scrum, ela provavelmente usa alguns de seus elementos e cerimônias, como o standup diário, o backlog do produto e as retrospectivas.

Sprint

Um sprint é um período de até um mês durante o qual um incremento potencialmente liberável é criado.

A duração típica do sprint é de uma ou duas semanas.

Um incremento é uma versão atualizada do software contendo todas as melhorias implementadas durante o sprint e incluindo melhorias feitas em todos os sprints anteriores.

No Scrum, os requisitos residem em um backlog de produtos. O atraso de produtos é “uma lista emergente e ordenada do que é necessário para melhorar o produto. Cada item de requisito no backlog é chamado de item de backlog do produto (PBI).

O Scrum impõe poucas restrições sobre o que um PBI pode ser ou como ele deve ser documentado.

O backlog do produto é um artefato dinâmico, com itens de requisitos adicionados, descartados, sequenciados novamente e refinados ao longo do ciclo de desenvolvimento.

No framework Scrum, o Product Owner (PO) é o responsável por maximizar o valor do produto desenvolvido pela equipe.

Ele atua como representante dos stakeholders e dos clientes, definindo claramente o que deve ser desenvolvido e em qual ordem de prioridade. Para isso, o PO gerencia o Product Backlog, organizando e priorizando os itens de acordo com o valor de negócio, necessidades do usuário e objetivos estratégicos da organização.

Além disso, o Product Owner esclarece requisitos, toma decisões sobre funcionalidades e garante que a equipe de desenvolvimento compreenda o que precisa ser entregue, contribuindo para que o produto evolua de forma alinhada às expectativas do cliente e às metas do projeto.

No framework Scrum, o Scrum Master é o responsável por garantir que a equipe compreenda e aplique corretamente os princípios e práticas do Scrum, atuando como facilitador do processo, organizando eventos como reuniões diárias, planejamento da sprint e retrospectivas.

Além disso, o Scrum Master trabalha para identificar e remover impedimentos que possam atrapalhar o progresso da equipe, promovendo um ambiente colaborativo e produtivo.

Diferentemente de um gerente tradicional, o Scrum Master não exerce autoridade hierárquica sobre a equipe, mas atua como um líder servidor, apoiando o time para que ele se torne cada vez mais autônomo, eficiente e focado na entrega contínua de valor.

O definição de feito (DoD) é um conjunto de condições que devem ser satisfeitas para que uma unidade de requisitos seja considerada completamente implementada —ou “concluída”.

O DoD atua como um conjunto mínimo de condições que cada item no backlog deve cumprir; além disso, cada item do backlog individual também tem seus próprios critérios de aceitação.

Os DoDs comuns incluem que o requisito seja discutido com os usuários, codificado, testado e que o código seja verificado, refatorado e integrado.

O definição de feito (DoD) é um conjunto de condições que devem ser satisfeitas para que uma unidade de requisitos seja considerada completamente implementada —ou “concluída”.

O DoD atua como um conjunto mínimo de condições que cada item no backlog deve cumprir; além disso, cada item do backlog individual também tem seus próprios critérios de aceitação.

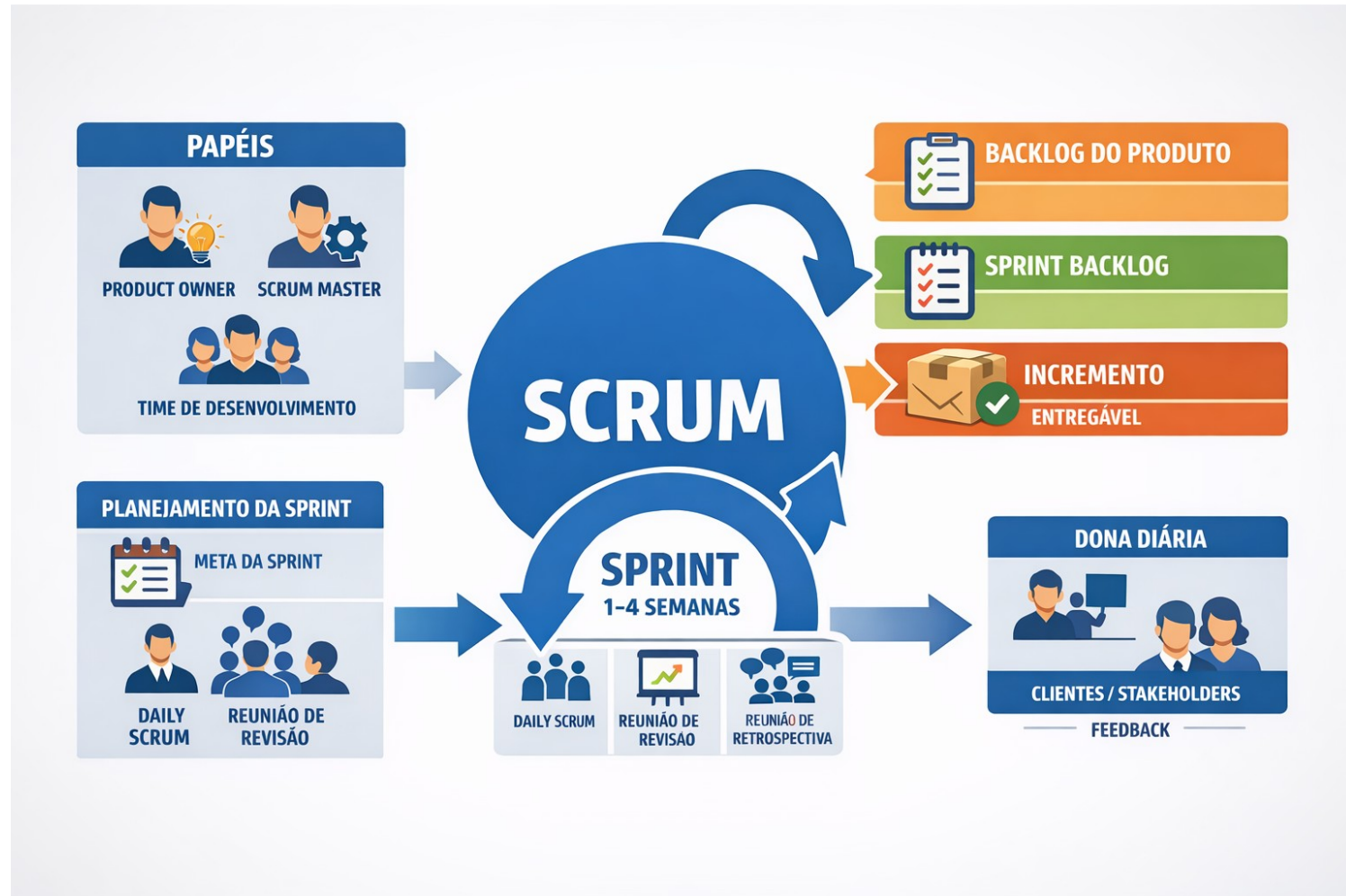
Os DoDs comuns incluem que o requisito seja discutido com os usuários, codificado, testado e que o código seja verificado, refatorado e integrado.

O scrum diário, comumente chamado de reunião diária, é uma reunião curta para planejar o trabalho do dia.

Deve ser realizado no mesmo horário todos os dias -geralmente ocorrendo pela manhã. Todos membros da equipe se reúnem para discutir o que fizeram em direção à meta do sprint desde a última reunião, o que farão até a próxima reunião e quaisquer impedimentos que prevejam.

O Guia Scrum é rigoroso quanto à aplicação de um limite de tempo de 15 minutos ao standup, mas você deve estender a reunião se a atualização exigir uma discussão mais longa.

Scrum



Programação Extrema

A Extreme Programming (XP) é uma metodologia ágil de desenvolvimento de software criada por Kent Beck na década de 1990 e apresentada em seu livro Extreme Programming Explained.

O XP é baseado em um processo iterativo e incremental, no qual o software é desenvolvido em pequenas iterações com entregas frequentes.

Assim como o Scrum, o XP utiliza ciclos de trabalho com tempo definido (timebox), porém se diferencia por possuir maior foco em práticas técnicas de programação, como programação em par, integração contínua, testes automatizados e refatoração constante do código.

Outra característica importante é a realização de iterações muito curtas, geralmente de cerca de uma semana, além de incluir planejamento de médio prazo, como o planejamento trimestral, algo que não é explicitamente tratado no Scrum.

Essas práticas ajudam a melhorar a qualidade do software e a capacidade da equipe de responder rapidamente a mudanças nos requisitos.

As práticas primárias do XP incluem o seguinte:

- Programação em pares—dois programadores trabalhando juntos
- Sentados juntos como uma equipe em um espaço aberto sem cubículos
- Trabalho energizado—horário de trabalho sustentável, evitando horas extras regulares
- Programação Test-first—escrevendo o teste antes do código
- Integração contínua (CI)
- Design incremental — projetar para as necessidades atuais e expandir o design à medida que o negócio cresce
- Testes automatizados



Processo Racional Unificado (RUP)

O Rational Unified Process (RUP) é um processo iterativo-incremental desenvolvido pela Rational Corporation no final da década de 1990 e é baseado no processo objetório. Objectory foi criado por Ivar Jacobsen no final da década de 1980.

No RUP, análise, design, codificação e testes ocorrem durante todas as fases do desenvolvimento -embora o esforço relativo varie ao longo do tempo.

O ciclo de vida do RUP começa com um curto início fase durante a qual o caso de negócios e o modelo de caso de uso de alto nível são criados, e protótipos e provas de conceito são construídos.

O próximo é um elaboração fase durante a qual a maioria (muitas vezes até 80%) dos requisitos são analisados antecipadamente e a arquitetura é testada e estabelecida.

Nesta fase, a maioria dos cenários de casos de uso são especificados.

É seguido por um construção fase durante a qual as atividades restantes de análise, projeto e implementação ocorrem de forma iterativa e incremental ao longo de uma série de iterações.

Finalmente, a transição ocorre a fase durante a qual a solução é migrada para a produção.

O Framework RUP

Rational Unified Process



Startup Lean

Lean startup foi desenvolvido por Eric Ries e descrito em seu livro A Startup Lean.

A abordagem lean startup é baseada em aprendizado validado, em que as decisões de investimento são baseadas no feedback de experimentos controlados no mercado.

Apesar do nome, a abordagem não se aplica apenas a organizações habitualmente consideradas startups.

Também pode se referir a empresas de telecomunicações estabelecidas, seguradoras e até mesmo agências governamentais.

O importante é que a organização esteja desenvolvendo um produto ou serviço inovador.

As práticas de lean startup incorporadas nas orientações de análise deste livro incluem o seguinte:

- Produto mínimo viável (MVP): A versão mínima de um produto necessária para facilitar o aprendizado no mercado
- Métricas acionáveis: métricas que fornecem orientação acionável sobre onde investir em seguida
- Hipóteses de salto de fé: teorias não comprovadas sobre o valor e o crescimento de um produto que precisam ser testadas rapidamente

LEAN STARTUP

Produto Mínimo Viável (MVP)

Versão Básica do Produto

Testar & Aprender



O mínimo para validar no mercado

Métricas Acionáveis

Dados que Guiam



• Medir, Aprender, Ajustar

Hipóteses de Salto de Fé

Suposições a Testar

Ideias
Arriscadas



• Teorias não Comprovadas

Construir → Medir → Aprender

Qual escolher ?

O Scrum é amplamente utilizado por sua simplicidade, organização em sprints e clareza de papéis, porém pode deixar lacunas em práticas técnicas de desenvolvimento.

O XP, por outro lado, complementa esse aspecto ao enfatizar qualidade de código, testes automatizados e programação em par, mas exige maior disciplina técnica da equipe.

O RUP oferece uma estrutura mais completa e documentada, com fases bem definidas e forte foco em arquitetura e análise antecipada, sendo útil em projetos complexos ou corporativos; entretanto, pode ser considerado mais pesado e burocrático em comparação com métodos ágeis.

Já a abordagem do Lean Startup incentiva experimentação rápida, validação de hipóteses e uso de métricas para orientar decisões, sendo muito adequada para inovação e desenvolvimento de novos produtos, embora possa não oferecer tantos detalhes sobre gestão do desenvolvimento técnico.

Assim, não existe um modelo universalmente melhor. A escolha depende de fatores como tamanho da equipe, maturidade técnica, tipo de produto, nível de incerteza do projeto e cultura organizacional.

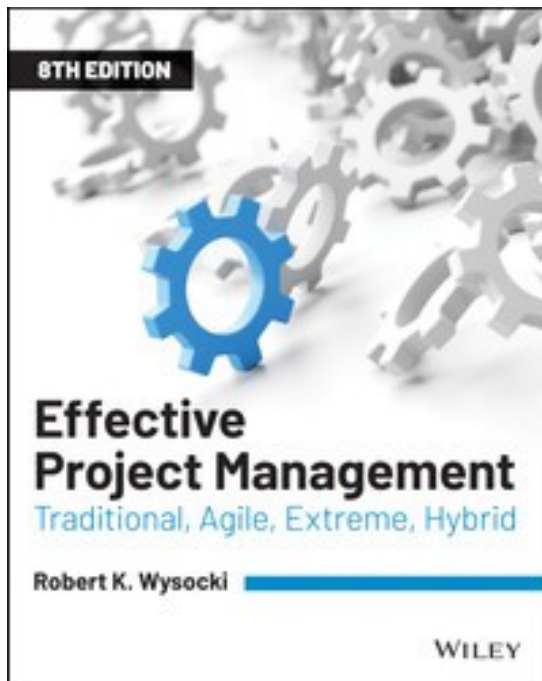
Em muitos casos, equipes combinam práticas de diferentes abordagens - por exemplo, usando Scrum para gestão do trabalho, XP para práticas de engenharia e Lean Startup para validação de ideias.

O mais importante é que o método adotado apoie a colaboração da equipe, facilite a entrega contínua de valor e possa ser adaptado ao contexto específico do projeto.

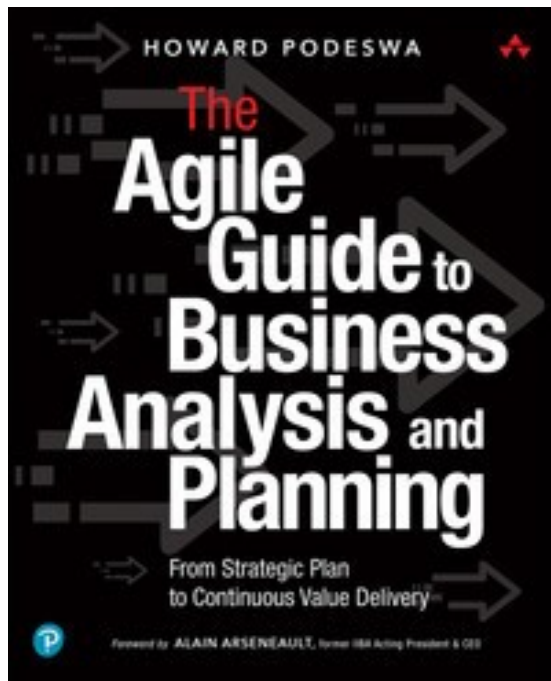
Dúvidas

Prof. Orlando Saraiva Júnior
orlando.nascimento@fatec.sp.gov.br

Effective Project Management. WYSOCKI, Robert K. 8. ed.
Hoboken, NJ: Wiley, 2019.



PODESWA, Howard. The Agile Guide to Business Analysis and Planning: From Strategic Plan to Continuous Value Delivery. Boston: Addison-Wesley Professional, 2021. 800 p.



Leitura do site indicado nos grupos de Whatsapp